1.6. Reconocimiento y evaluación de las herramientas de programación y prueba sobre clientes Web (Criterio 1.6)

- **Resultado de Aprendizaje Asociado: RA1** (*Selecciona las arquitecturas y tecnologías de programación sobre clientes Web, identificando y analizando las capacidades y características de cada una*).

- **Criterio Curricular Oficial: CE 1.f** (Se han reconocido y evaluado las herramientas de programación y prueba sobre clientes web).

- **Ponderación del Criterio:** (**16,67% del RA1** | **0,833% sobre la calificación final del módulo**).

El desarrollo profesional en JavaScript requiere superar el uso de editores de texto elementales y adoptar entornos de trabajo que integren asistentes de código, depuradores interactivos, gestores de dependencias y sistemas de control de versiones. El técnico en desarrollo de aplicaciones web debe evaluar y seleccionar las herramientas adecuadas en función de la envergadura del proyecto, la infraestructura disponible y el flujo de trabajo en equipo.



A. Herramientas Locales: De Editores Básicos a IDEs Avanzados

Para escribir código JavaScript es técnicamente suficiente un editor de texto plano sin formato (como Notepad en Windows o gedit en Linux). Sin embargo, en un entorno de desarrollo empresarial esta práctica es inviable debido a la falta de herramientas que automaticen la verificación de la sintaxis y la gestión de proyectos.

Ranking de Editores Top para JavaScript / TypeScript:

El ranking mundial de entornos y editores más utilizados específicamente para **JavaScript y TypeScript** según la encuesta global de Stack Overflow Developer Survey:

- **1. Visual Studio Code (75.9% de uso global)**
  - **Su rol en JS/TS:** Es el estándar absoluto de la industria. Más del **80% de los desarrolladores frontend** lo tienen como su herramienta principal. Cuenta con soporte nativo de fábrica para TypeScript (ya que el propio editor está programado en TS) y extensiones obligatorias de ecosistema como *ESLint, Prettier* y los *React/Vue Snippets*.
- **2. Notepad++ (27.4% de uso global)**
  - **Su rol en JS/TS:** Aunque no es un entorno para armar una aplicación moderna compleja (como una app de Next.js), sigue apareciendo extremadamente alto en las métricas globales porque miles de desarrolladores lo usan en Windows para la edición rápida de scripts sueltos de JavaScript, manipulación veloz de archivos .json gigantescos o tareas ligeras de automatización sin consumir recursos.
- **3. Vim / Neovim (38.3% de uso combinado)**
  - *Vim: 24.3% | Neovim: 14%*
  - **Su rol en JS/TS:** Es el entorno favorito de los desarrolladores avanzados y administradores de servidores que obtienen con Neovim el mismo autocompletado y tipado inteligente de TypeScript que ofrece VS Code, pero corriendo directo en la terminal a máxima velocidad.
- **4. Cursor (17.9% de uso global y subiendo)**
  - **Su rol en JS/TS:** Es la herramienta de Inteligencia Artificial que más rápido ha escalado en los rankings. Al ser un clon exacto de VS Code, se ha vuelto sumamente popular entre desarrolladores de JavaScript porque permite usar la IA nativa para generar componentes interactivos completos de interfaces web o refactorizar archivos TypeScript complejos con instrucciones simples en lenguaje natural.
- **5. Los IDEs de JetBrains (15.1% combinados en web)**
  - *WebStorm: 7.6%*
  - **Su rol en JS/TS:** WebStorm es considerado el Rolls-Royce de los IDEs para JavaScript. Su porcentaje global parece menor porque es una herramienta tradicionalmente comercial de pago, pero en entornos profesionales y corporativos es muy cotizado debido a que su motor de **refactorización de código** y detección de rutas rotas en JavaScript/TypeScript es el más inteligente y seguro del mercado. *(Nota: JetBrains recientemente liberó una versión totalmente gratuita de WebStorm para uso no comercial, lo que está impulsando su adopción).*

La relación entre VS Code y VSCodium

Aunque ambos editores se ven e内部mente actúan de forma idéntica, la diferencia radica en la privacidad y la licencia de distribución:

- Visual Studio Code **(Propietario de Microsoft):** El código fuente base es libre (llamado *Code - OSS*), pero el instalador final que descargas de Microsoft incluye una licencia comercial. Esto significa que contiene **telemetría** (rastreadores que envían datos de uso a Microsoft) y restricciones para usar ciertas extensiones oficiales en otros editores.
- VSCodium **(100% Libre y Comunitario):** Es un proyecto independiente que toma exactamente el mismo código fuente libre de Microsoft y lo compila de forma limpia. **Elimina por completo toda la telemetría, el rastreo de datos y las licencias comerciales**. Es la opción preferida por los defensores del software libre y la privacidad extrema.

Editores Independientes con IA Nativa

- Windsurf **(De Codeium):** Es actualmente el **competidor más fuerte y directo de Cursor**. Al igual que este, es un clon (*fork*) de VS Code, por lo que mantiene la misma interfaz y compatibilidad con tus extensiones. Destaca por su modo agente llamado *Cascade*, el cual es capaz de analizar de forma autónoma bases de código complejas, ejecutar comandos en la terminal, leer errores de JavaScript y corregir bugs de manera independiente. Tiene un plan gratuito muy generoso.
- Void**:** Es una alternativa **100% de código abierto** construida por la comunidad para competir contra el modelo comercial cerrado de Cursor. Su gran enfoque es la **privacidad total**: el editor es completamente gratuito y está diseñado para que conectes tus propias claves de API (de OpenAI, Anthropic, Gemini) o utilices modelos de IA locales que corran en tu propia computadora.
- Zed**:** El editor de rendimiento extremo escrito en **Rust** (que no consume apenas RAM y usa la GPU). Es completamente gratuito y de código abierto. Cuenta con funciones de IA integradas de forma nativa en su interfaz gráfica donde puedes configurar tu propia API Key para generar código a una velocidad instantánea.

Características técnicas fundamentales para la elección de un entorno profesional

- **Código abierto (*Open-source*) y gratuidad:** Garantiza que la comunidad de programadores audite el código, reporte fallos y publique mejoras de forma continua, solucionando incidencias con mayor rapidez que en aplicaciones propietarias cerradas.
- **Arquitectura modular:** Permite activar, desactivar o reemplazar componentes internos del editor según los gustos o necesidades del desarrollador.
- **Gestor de paquetes integrado (*Package Manager*):** Mecanismo de línea de comandos o interfaz visual para registrar, instalar, actualizar y eliminar librerías, extensiones y temas visuales de forma desatendida.
- **Autocompletado predictivo:** Asistente que analiza las variables, funciones y métodos del lenguaje mientras se escribe, minimizando fallos tipográficos y acelerando la codificación.

    - **Sistema de paneles múltiples:** Organización visual del espacio de trabajo en paneles divididos para comparar y editar varios archivos (HTML, CSS y JS) simultáneamente.
    - **Soporte y canales comunitarios:** Apoyo técnico distribuido en foros y plataformas colaborativas como **Slack** o redes sociales.

B. Integración con Sistemas de Control de Versiones (Git y GitHub)

El desarrollo en equipo exige registrar el historial de cambios, gestionar ramas de características y coordinar modificaciones concurrentes:

- **Git:** Sistema de control de versiones distribuido que rastrea cada modificación realizada en los archivos del proyecto a lo largo del tiempo.
- **GitHub:** Plataforma en la nube para alojar repositorios Git, facilitando la revisión de código por pares (*pull requests*), el seguimiento de incidencias (*issues*) y la integración continua.
- **Integración en el IDE:** Editores Visual Studio Code integran paneles nativos de Git que permiten confirmar cambios (*commits*), alternar entre ramas y resolver conflictos sin salir del entorno de edición.

C. Entornos de Programación y Prueba Online

Cuando se necesita probar fragmentos de código de forma inmediata sin configurar un entorno local, o cuando se trabaja desde dispositivos con restricciones de instalación, los IDEs en la nube ofrecen una alternativa funcional:

- **Coding Ground (Tutorialspoint):**
    - Plataforma accesible vía navegador web (https://www.tutorialspoint.com/online_javascript_editor.php).
    - Proporciona un editor con resaltado de sintaxis, visualización previa (*Preview*) y consola de ejecución simultánea.
    - Permite gestionar múltiples ficheros dentro de un mismo proyecto, así como descargar el código generado al equipo local o importar archivos externos.
- **Otras plataformas en la nube:** Soluciones como CodeSandbox, StackBlitz o JSFiddle permiten evaluar librerías y componentes sin requerir instalación previa. Permiten arrancar proyectos de React, Angular o Vue directamente desde el navegador en un par de segundos.

D. Herramientas de Prueba y Depuración del Navegador (*DevTools*)

El navegador web integra su propio conjunto de herramientas de diagnóstico técnico, accesibles de forma universal pulsando la tecla **F12** o mediante la combinación Ctrl + Shift + I:

- **Panel Consola (*Console*):**
    - Permite interactuar directamente con el motor de JavaScript en tiempo real.
    - Muestra las salidas emitidas por el código mediante console.log() y resalta en color rojo las excepciones y errores no capturados durante la ejecución.
- **Panel Fuentes (*Sources / Debugger*):**

    - Permite examinar los ficheros .js descargados y establecer **puntos de interrupción (*breakpoints*)** en líneas concretas.
    - Cuando la ejecución alcanza un punto de interrupción, el navegador congela la ejecución del script, permitiendo al desarrollador inspeccionar el valor de las variables en memoria paso a paso y analizar la pila de llamadas (*Call Stack*).
- **Panel Red (*Network*):**
    - Supervisa todas las peticiones HTTP que realiza la página web (ficheros HTML, hojas de estilo CSS, scripts .js, imágenes o peticiones asíncronas de datos).
    - Permite comprobar el código de respuesta del servidor (200 OK, 404 Not Found, 500 Server Error), el tiempo exacto de transferencia y el tamaño de los recursos descargados.

E. Criterios de Evaluación y Selección de Herramientas

Para seleccionar el software de desarrollo en un proyecto web, se aplican los siguientes factores técnicos:

Parámetro de Decisión	**Editor Ligero / Online**	**IDE Completo / Avanzado**
Escenario de uso idóneo	Pruebas de concepto rápidas, corrección puntual de errores, equipos con hardware limitado.	Proyectos profesionales medianos y grandes, aplicaciones basadas en frameworks (React, Angular).
Consumo de recursos	Mínimo; funciona en cualquier navegador web.	Medio-alto; requiere memoria RAM y almacenamiento local para indexar el proyecto.
Control de versiones	Limitado a exportar o descargar archivos sueltos.	Integración profunda con Git, ramas, control de diferencias visuales y GitHub.
Personalización	Escasa o nula; depende de la plataforma web.	Elevada; personalizable mediante gestores de paquetes y extensiones de la comunidad.

F. Actividades Prácticas y Ejercicios del Criterio 1.6

Actividad de Configuración de Entorno:

Instala un editor modular avanzado (como Visual Studio Code o Cursor). Para que tu experiencia programando solo con JavaScript en VS Code o Cursor sea excelente, instala estas **3 extensiones específicas adicionales** y configurar una herramienta clave:

Live Server (Imprescindible para Frontend)

Cuando trabajas con JavaScript puro, necesitas enlazar tu archivo .js a un archivo index.html.

- **Qué hace:** Crea un servidor web local en tu computadora con un solo clic. Cada vez que hagas un cambio en tu código JavaScript u HTML y guardes, **la página del navegador se recargará automáticamente**. Ya no tendrás que abrir el archivo desde tu carpeta ni presionar *F5* constantemente.

Quokka.js (Tu bloc de notas interactivo)

Es una herramienta brutal para aprender y experimentar con JavaScript.

- **Qué hace:** Ejecuta tu código JavaScript **en tiempo real mientras lo vas escribiendo**, directamente dentro del propio editor. Si creas una función o una operación matemática, Quokka te muestra el resultado flotando justo al lado de la línea de código, sin necesidad de abrir el navegador ni la terminal. Es ideal para probar lógica rápido.

Error Lens (Detección visual inmediata)

- **Qué hace:** Normalmente, cuando te equivocas en JavaScript (por ejemplo, te falta un paréntesis o escribes mal el nombre de una variable), el editor te pone una línea ondulada roja muy pequeña debajo del error. Error Lens toma ese mensaje de error y lo **escribe en texto completo con fondo rojo directamente al final de la línea**, para que veas instantáneamente qué está fallando sin tener que pasar el cursor por encima.